capstone_MEDICAl_INSURANCE_FRAUD_DETECTION SYSTEM-1.pdf
claim associated with patient.csv
claim submitted_by prescriber.csv
claim.csv
specialties-subspecialties.csv
subspecialties-prescriber.csv
You are a senior Machine Learning Engineer and Graph Data Scientist. Your task is to design and implement a complete production-ready Machine Learning project titled: "Medical Insurance Fraud Detection System using Machine Learning and Neo4j Graph Database" The project must strictly prioritize the objectives below while improving architecture if needed. ---------- PROJECT OBJECTIVES (HIGH PRIORITY) ---------- 1. Study and analyze medical insurance claim data 2. Detect fraudulent claims using machine learning techniques 3. Model relationships between doctors, claims, and specialties using Neo4j 4. Identify abnormal and suspicious claim patterns 5. Assist investigators by providing explainable fraud insights The system must detect fraud patterns such as: - doctors submitting excessive claims - specialty mismatch - repeated claims - unrealistic treatment schedules - abnormal provider behaviour ---------- DATASETS ---------- The project uses the following CSV datasets: 1. claim.csv 2. claim_associated_with_patient.csv 3. claim_submitted_by_prescriber.csv 4. specialties-subspecialties.csv 5. subspecialties-prescriber.csv These datasets describe relationships between: Patient → Claim Doctor/Prescriber → Claim Doctor → Specialty Specialty → Subspecialty ---------- SYSTEM ARCHITECTURE ---------- Design a hybrid architecture: DATA PIPELINE CSV files ↓ Data Cleaning + Feature Engineering ↓ Graph Construction (Neo4j) ↓ Graph Feature Extraction ↓ Machine Learning Model ↓ Fraud Prediction ↓ Graph-based Investigation Dashboard ---------- GRAPH DATABASE DESIGN (NEO4J) ---------- Use Neo4j to represent relationships. Node Types: Patient Doctor Claim Specialty Subspecialty Relationships: (Patient)-[:HAS_CLAIM]->(Claim) (Doctor)-[:SUBMITTED]->(Claim) (Doctor)-[:SPECIALIZED_IN]->(Specialty) (Specialty)-[:HAS_SUBSPECIALTY]->(Subspecialty) Derived Relationships: (Doctor)-[:TREATED]->(Patient) ---------- GRAPH ANALYTICS ---------- Use Neo4j Graph Data Science (GDS) algorithms. Compute graph features such as: doctor_claim_count patient_claim_count doctor_patient_degree claim_frequency doctor_specialty_consistency shared_patient_ratio provider_network_density Also compute graph algorithms: PageRank Node Degree Betweenness Centrality Community Detection (Louvain) These graph features should be merged with ML features. ---------- FEATURE ENGINEERING ---------- Create three categories of features: 1. Claim Features claim_amount claim_frequency claim_time_gap duplicate_claim_flag 2. Provider Features claims_per_doctor patients_per_doctor specialty_mismatch multi_location_claims 3. Graph Features doctor_centrality provider_cluster_size patient_provider_overlap community_risk_score ---------- MACHINE LEARNING MODEL ---------- Choose models that work well on tabular + graph features and can run on a 4GB GPU. Primary model: LightGBM Classifier Secondary model: Random Forest Optional ensemble: LightGBM + RandomForest VotingClassifier Reason: High performance Low memory usage Handles tabular data well Explainable with SHAP ---------- TRAINING SETTINGS (IMPORTANT) ---------- Must run on 4GB GPU. Batch size: 64 Learning rate: 0.05 Number of estimators: 300 Max depth: 8 Early stopping: enabled Train-test split: 80 / 20 Use stratified sampling. ---------- MODEL CHECKPOINTING (NEW SECTION) ---------- Implement a checkpoint system to save model parameters during training. Requirements: • Save model checkpoints every 10 boosting iterations. • Store checkpoints in the directory: models/checkpoints/ Example filenames: fraud_model_iter_10.pkl fraud_model_iter_20.pkl fraud_model_iter_30.pkl • Always save the best performing model based on validation AUC as: models/fraud_model_best.pkl • Allow training to resume from the latest checkpoint if interrupted. • Use joblib or pickle for saving model states. • Log metrics (loss, AUC, recall) for every checkpoint. • Maintain a training log file: logs/training_log.csv ---------- MODEL EXPLAINABILITY ---------- Use SHAP to explain predictions. Output: Top features causing fraud Doctor network risk score Suspicious graph connections Example explanation: "Claim flagged because doctor has unusually high claim volume and treats patients outside specialization." ---------- FRAUD PATTERN DETECTION RULES ---------- Implement rule-based detectors using Neo4j Cypher: 1. Too many claims per doctor 2. Doctor treating outside specialty 3. Multiple claims same patient same day 4. Doctor appearing in many fraud communities ---------- PROJECT FOLDER STRUCTURE ---------- medical_fraud_detection/ data/ raw/ processed/ notebooks/ EDA.ipynb graph_analysis.ipynb src/ data_processing.py feature_engineering.py neo4j_loader.py graph_features.py train_model.py evaluate_model.py explainability.py fraud_rules.py models/ fraud_model.pkl models/checkpoints/ logs/ config/ config.yaml dashboard/ streamlit_app.py ---------- IMPLEMENTATION REQUIREMENTS ---------- Use Python. Libraries: pandas numpy scikit-learn lightgbm neo4j py2neo networkx shap matplotlib streamlit ---------- DASHBOARD ---------- Build a Streamlit dashboard that shows: fraud probability suspicious doctors claim network visualization top fraud features graph connections ---------- EXPECTED OUTPUTS ---------- 1. Fraud prediction model 2. Neo4j graph database 3. Fraud investigation dashboard 4. Graph queries to detect suspicious behaviour 5. Explainable fraud predictions ---------- PERFORMANCE METRICS ---------- Evaluate using: Accuracy Precision Recall F1 Score ROC AUC Fraud detection should prioritize high recall. ---------- DELIVERABLES ---------- Generate: 1. Full project code 2. Neo4j schema 3. Graph feature engineering 4. ML training pipeline 5. Fraud detection rules 6. Streamlit dashboard 7. README explaining architecture 8. Setup instructions ---------- Focus on: Explainability Practical deployment Neo4j graph insights ML + Graph hybrid detection Low memory usage Avoid heavy GNN models that require large GPUs. Produce clean, modular, production-quality code.

I’m setting up the project in this workspace and first gathering the dataset/schema details from the CSVs and the existing files so we can build the pipeline around the real columns rather than guessing.

Background terminal finished with pwd

Explored 1 list

Background terminal finished with python3 - <<'PY' from pathlib import Path import subprocess, textwrap pdf = Path('/Users/ashish/Labcode/Projects/Datacapstone/capstone_MEDICAl_INSURANCE_FRAUD_DETECTION SYSTEM-1.pdf') if not pdf.exists(): print('PDF_NOT_FOUND') raise SystemExit for cmd in (["pdftotext", str(pdf), "-"], ["python3", "-c", "import fitz,sys; doc=fitz.open(sys.argv[1]); print(doc[0].get_text())", str(pdf)]): try: out = subprocess.check_output(cmd, stderr=subprocess.STDOUT, text=True, timeout=20) print(out[:5000]) break except Exception as e: last = e else: print(f'FAILED_TO_READ_PDF: {last}') PY

Background terminal finished with python3 - <<'PY' import pandas as pd from pathlib import Path files = [Path('/Users/ashish/Labcode/Projects/Datacapstone/data/claim.csv'), Path('/Users/ashish/Labcode/Projects/Datacapstone/data/claim associated with patient.csv'), Path('/Users/ashish/Labcode/Projects/Datacapstone/data/claim submitted_by prescriber.csv'), Path('/Users/ashish/Labcode/Projects/Datacapstone/data/specialties-subspecialties.csv'), Path('/Users/ashish/Labcode/Projects/Datacapstone/data/subspecialties-prescriber.csv'),] for path in files: print(f'FILE: {path.name}') df = pd.read_csv(path) print('shape:', df.shape) print('columns:', list(df.columns)) print(df.head(3).to_string(index=False)) print('-'*80) PY

I have the raw CSV inspection running now. The current workspace looks empty, so I’ll scaffold the project from scratch once I finish mapping the real dataset fields.

The data is leaner than the requested feature list: we have claim diagnosis/time data plus patient, prescriber, specialty, and subspecialty links, but no explicit fraud label, amount, or location columns. I’m digging into joins and anomaly opportunities next so the implementation uses solid derived features and handles missing business fields cleanly.

Background terminal finished with python3 - <<'PY' import pandas as pd from pathlib import Path base = Path('/Users/ashish/Labcode/Projects/Datacapstone/data') claim = pd.read_csv(base/'claim.csv') cp = pd.read_csv(base/'claim associated with patient.csv') cd = pd.read_csv(base/'claim submitted_by prescriber.csv') sp = pd.read_csv(base/'specialties-subspecialties.csv') pp = pd.read_csv(base/'subspecialties-prescriber.csv') print('claim unique ids', claim['id'].nunique()) print('claim-patient unique claims/patients', cp['Claim'].nunique(), cp['Patient'].nunique()) print('claim-doctor unique claims/doctors', cd['Claim'].nunique(), cd['Prescriber'].nunique()) print('specialties/subspecialties', sp['Specialty'].nunique(), sp['SubSpecialty'].nunique()) print('doctor-subspecialties', pp['Prescriber'].nunique(), pp['SubSpecialty'].nunique()) print('claims missing patient link', len(set(claim['id']) - set(cp['Claim']))) print('claims missing doctor link', len(set(claim['id']) - set(cd['Claim']))) print('patients with >1 claim sample') print(cp['Patient'].value_counts().head(10).to_string()) print('doctors with >1 claim sample') print(cd['Prescriber'].value_counts().head(10).to_string()) print('doctor specialty counts sample') print(pp.groupby('Prescriber')['SubSpecialty'].nunique().sort_values(ascending=False).head(10).to_string()) print('duplicate claim-patient rows', cp.duplicated().sum()) print('duplicate claim-doctor rows', cd.duplicated().sum()) PY

Background terminal finished with python3 - <<'PY' import pandas as pd from pathlib import Path base = Path('/Users/ashish/Labcode/Projects/Datacapstone/data') claim = pd.read_csv(base/'claim.csv') cp = pd.read_csv(base/'claim associated with patient.csv') cd = pd.read_csv(base/'claim submitted_by prescriber.csv') pp = pd.read_csv(base/'subspecialties-prescriber.csv') sp = pd.read_csv(base/'specialties-subspecialties.csv') merged = claim.rename(columns={'id':'Claim'})\ .merge(cp, on='Claim', how='left')\ .merge(cd, on='Claim', how='left')\ .merge(pp, on='Prescriber', how='left')\ .merge(sp, on='SubSpecialty', how='left') merged['time'] = pd.to_datetime(merged['time'], errors='coerce') print('merged shape', merged.shape) print('null rates') print((merged.isna().mean().sort_values(ascending=False).head(10)*100).round(2).to_string()) print('date range', merged['time'].min(), merged['time'].max()) print('rows with no specialty', merged['Specialty'].isna().sum()) print('sample rows') print(merged.head(5).to_string(index=False)) print('same patient same day duplicate counts') dups = merged.groupby(['Patient','time']).size().sort_values(ascending=False).head(10) print(dups.to_string()) print('same doctor same patient same day counts') dps = merged.groupby(['Prescriber','Patient','time']).size().sort_values(ascending=False).head(10) print(dps.to_string()) PY

Background terminal finished with python3 - <<'PY' import pandas as pd from pathlib import Path base = Path('/Users/ashish/Labcode/Projects/Datacapstone/data') claim = pd.read_csv(base/'claim.csv') for col in ['Code Group Title', 'ICD -9', 'ICD-10', 'ICD-9 Description', 'ICD-10 Description']: print('\n', col) print(claim[col].value_counts(dropna=False).head(15).to_string()) PY

I've confirmed the key constraint: there's no ground-truth fraud label in the provided data, and `claim.csv` is claim-line level while the relationship tables are claim-level. I'm turning that into a production-friendly hybrid design where rules and graph anomalies generate weak labels, then the ML layer learns a fraud-risk model on top.

Background terminal finished with python3 - <<'PY' import pandas as pd from pathlib import Path base = Path('/Users/ashish/Labcode/Projects/Datacapstone/data') claim = pd.read_csv(base/'claim.csv') cp = pd.read_csv(base/'claim associated with patient.csv') cd = pd.read_csv(base/'claim submitted_by prescriber.csv') pp = pd.read_csv(base/'subspecialties-prescriber.csv') sp = pd.read_csv(base/'specialties-subspecialties.csv') merged = claim.rename(columns={'id':'Claim'}).merge(cp, on='Claim').merge(cd, on='Claim').merge(pp, on='Prescriber').merge(sp, on='SubSpecialty') # Specialty x code groups top associations ct = pd.crosstab(merged['Specialty'], merged['Code Group Title']) for spec in ct.index: top = ct.loc[spec].sort_values(ascending=False).head(5) print('\nSPECIALTY:', spec) print(top.to_string()) PY

Background terminal finished with python3 - <<'PY' mods = ['pandas','numpy','sklearn','lightgbm','neo4j','py2neo','networkx','shap','matplotlib','streamlit','yaml','joblib'] for m in mods: try: __import__(m) print(m, 'OK') except Exception as e: print(m, 'MISSING', e.__class__.__name__, str(e)) PY

Background terminal finished with python3 - <<'PY' import pandas as pd from pathlib import Path base = Path('/Users/ashish/Labcode/Projects/Datacapstone/data') claim = pd.read_csv(base/'claim.csv').rename(columns={'id':'Claim'}) cp = pd.read_csv(base/'claim associated with patient.csv') cd = pd.read_csv(base/'claim submitted_by prescriber.csv') merged = claim.merge(cp, on='Claim').merge(cd, on='Claim') merged['time'] = pd.to_datetime(merged['time']) # top doctor-patient claim pairs pairs = merged.groupby(['Prescriber','Patient']).size().sort_values(ascending=False).head(20) print('top doctor-patient pair counts') print(pairs.to_string()) # shortest gaps per doctor-patient merged = merged.sort_values(['Prescriber','Patient','time']) merged['gap'] = merged.groupby(['Prescriber','Patient'])['time'].diff().dt.days print('\nnon-null zero/negative/short gaps counts') print((merged['gap']<=0).sum(), (merged['gap']<=1).sum(), (merged['gap']<=7).sum()) print('\nsmallest gaps sample') print(merged.loc[merged['gap'].notna(), ['Claim','Prescriber','Patient','time','gap']].sort_values('gap').head(20).to_string(index=False)) PY

I've got the implementation shape now. Next I'm scaffolding the project tree and then I'll fill the core modules: data prep, graph features, weak-label rules, checkpointed training, evaluation, explainability, and the dashboard.

Background terminal finished with mkdir -p '/Users/ashish/Documents/New project/medical_fraud_detection/data/raw' '/Users/ashish/Documents/New project/medical_fraud_detection/data/processed' '/Users/ashish/Documents/New project/medical_fraud_detection/notebooks' '/Users/ashish/Documents/New project/medical_fraud_detection/src' '/Users/ashish/Documents/New project/medical_fraud_detection/models/checkpoints' '/Users/ashish/Documents/New project/medical_fraud_detection/logs' '/Users/ashish/Documents/New project/medical_fraud_detection/config' '/Users/ashish/Documents/New project/medical_fraud_detection/dashboard'

Background terminal finished with python3 - <<'PY' import pandas as pd from pathlib import Path claim = pd.read_csv(Path('/Users/ashish/Labcode/Projects/Datacapstone/data/claim.csv')) dup = claim[claim['id'].duplicated(keep=False)].sort_values('id') print('duplicate claim ids', dup['id'].nunique()) print(dup.head(20).to_string(index=False)) print('\nrows with time variation within claim id:', dup.groupby('id')['time'].nunique().gt(1).sum()) print('rows with code group variation within claim id:', dup.groupby('id')['Code Group Title'].nunique().gt(1).sum()) PY

I'm writing the shared config/runtime layer first so the rest of the modules can stay consistent and reusable. After that I'll fill the feature and training code against this foundation.

Created .gitignore
+16-0

Created config.yaml
+63-0

Created fraud_rules.cypher
+24-0

Created neo4j_schema.cypher
+32-0

Created requirements.txt
+13-0

Created __init__.py
+14-0

Created common.py
+229-0

Created data_processing.py
+189-0
Created feature_engineering.py
+205-0
Created fraud_rules.py
+244-0
Created graph_features.py
+277-0
The feature and graph layer is in place now. I'm wiring the training stack and dashboard next, including the checkpoint/resume flow and explainability outputs so the project is runnable end-to-end.
Created streamlit_app.py
+196-0
Created .gitkeep
+1-0
Created .gitkeep
+1-0
Created .gitkeep
+1-0
Created evaluate_model.py
+77-0
Created explainability.py
+153-0
Created neo4j_loader.py
+217-0
Created train_model.py
+588-0
I'm adding the user-facing pieces now: the README, setup flow, and starter notebooks. After that I'll run the pipeline in this environment and tighten any bugs the smoke test surfaces.
Created README.md
+222-0
Created .gitkeep
+1-0
Created EDA.ipynb
+69-0
Created graph_analysis.ipynb
+67-0
I'm doing a quick hardening pass before the smoke test: a few path/config details and training edge cases are worth fixing now so the end-to-end run is less brittle.
Edited feature_engineering.py
+4-4
Edited train_model.py
+6-4
Explored 1 list
Background terminal finished with python3 -m compileall src dashboard
The project compiles cleanly. I'm running the actual pipeline now against your datasets so we can catch real data-shape issues and leave you with generated outputs, not just scaffolding.
Background terminal finished with python3 src/data_processing.py
Background terminal finished with which python3
Background terminal finished with python3 - <<'PY' import sys print(sys.executable) try: import pandas print('pandas ok', pandas.__version__) except Exception as exc: print('pandas fail', repr(exc)) PY
The code is fine, but the current shell is picking a bare Python 3.13 interpreter without the data-science packages we profiled earlier. I'm locating the right interpreter so I can run the smoke test with the installed stack instead of failing on imports.
Background terminal finished with which -a python
Background terminal finished with zsh -lc 'for p in $(which -a python3 python 2>/dev/null | awk '!seen[$0]++'); do echo PY:$p; "$p" - <<"PY" import sys print(sys.executable) try: import pandas print("PANDAS", pandas.__version__) except Exception as e: print("NO", type(e).__name__, e) PY done'
Background terminal finished with which -a python3
Background terminal finished with /usr/local/bin/python3 - <<'PY' import sys print(sys.executable) try: import pandas print('PANDAS', pandas.__version__) except Exception as e: print('NO', type(e).__name__, e) PY

Background terminal finished with /opt/anaconda3/bin/python3 - <<'PY' import sys print(sys.executable) try: import pandas print('PANDAS', pandas.__version__) except Exception as e: print('NO', type(e).__name__, e) PY

Background terminal finished with /opt/anaconda3/bin/python - <<'PY' import sys print(sys.executable) try: import pandas print('PANDAS', pandas.__version__) except Exception as e: print('NO', type(e).__name__, e) PY

Background terminal finished with /opt/anaconda3/bin/python src/feature_engineering.py

The first runtime issue is just Python path ergonomics: the modules compile, but direct script execution from `src/...` needs the project root on `sys.path`. I'm patching that once across the CLI modules so the commands in the README work as written.

Edited data_processing.py
+5-1

Edited feature_engineering.py
+5-0

Edited fraud_rules.py
+5-1

Edited graph_features.py
+5-1

Explored 4 files

Edited train_model.py
+5-1

Edited evaluate_model.py
+5-1

Edited explainability.py
+5-1

Edited neo4j_loader.py
+5-1

Background terminal finished with /opt/anaconda3/bin/python src/feature_engineering.py

The first data-driven bug is in the rolling window helper: grouped indices aren't zero-based, so direct positional indexing blew up on the real dataset. I'm switching that helper to use a plain NumPy array, which also removes the deprecation warning.

Edited feature_engineering.py
+1-1

Background terminal finished with /opt/anaconda3/bin/python src/feature_engineering.py

Background terminal finished with /opt/anaconda3/bin/python src/train_model.py

Training hit an environment-specific OpenMP shared-memory issue, not a modeling bug. I'm reducing parallelism in the trainer and pinning thread counts so it stays stable in constrained runtimes like this one.

Edited train_model.py
+6-1

Explored 1 file

Edited train_model.py
+5-4

Explored 3 lists

Background terminal finished with OMP_NUM_THREADS=1 OPENBLAS_NUM_THREADS=1 MKL_NUM_THREADS=1 NUMEXPR_NUM_THREADS=1 /opt/anaconda3/bin/python src/train_model.py

Background terminal finished with OMP_NUM_THREADS=1 OPENBLAS_NUM_THREADS=1 MKL_NUM_THREADS=1 NUMEXPR_NUM_THREADS=1 /opt/anaconda3/bin/python - <<'PY' import numpy as np from sklearn.ensemble import GradientBoostingClassifier X = np.random.rand(100,5) y = (X[:,0] > 0.5).astype(int) model = GradientBoostingClassifier(n_estimators=10, max_depth=3) model.fit(X,y) print('ok', model.predict_proba(X[:3])) PY

Background terminal finished with OMP_NUM_THREADS=1 OPENBLAS_NUM_THREADS=1 MKL_NUM_THREADS=1 NUMEXPR_NUM_THREADS=1 /opt/anaconda3/bin/python - <<'PY' import numpy as np from sklearn.linear_model import LogisticRegression X = np.random.rand(100,5) y = (X[:,0] > 0.5).astype(int) model = LogisticRegression(max_iter=100) model.fit(X,y) print('ok', model.predict_proba(X[:3])) PY

Background terminal finished with OMP_NUM_THREADS=1 OPENBLAS_NUM_THREADS=1 MKL_NUM_THREADS=1 NUMEXPR_NUM_THREADS=1 /opt/anaconda3/bin/python - <<'PY' import pandas as pd print('pandas ok', pd.__version__) PY

The remaining blocker is environment-specific: any `scikit-learn` fit in the available Anaconda interpreter is crashing inside its OpenMP runtime. I'm probing that runtime directly to see if there's a safe local workaround before I fall back to documenting the limitation.

Explored 1 search

Background terminal finished with /opt/anaconda3/bin/python - <<'PY' import ctypes.util print(ctypes.util.find_library('iomp5')) print(ctypes.util.find_library('omp')) PY

Background terminal finished with /opt/anaconda3/bin/python - <<'PY' import sklearn, os print(sklearn.__file__) PY

Background terminal finished with strings /opt/anaconda3/lib/libiomp5.dylib | rg 'SHM|KMP_' | head -n 200

Background terminal finished with KMP_DUPLICATE_LIB_OK=TRUE KMP_INIT_AT_FORK=FALSE KMP_HANDLE_SIGNALS=0 OMP_NUM_THREADS=1 OPENBLAS_NUM_THREADS=1 MKL_NUM_THREADS=1 NUMEXPR_NUM_THREADS=1 /opt/anaconda3/bin/python - <<'PY' import numpy as np from sklearn.linear_model import LogisticRegression X = np.random.rand(100,5) y = (X[:,0] > 0.5).astype(int) model = LogisticRegression(max_iter=100) model.fit(X,y) print('ok', model.predict_proba(X[:3])) PY

I can likely get a fully runnable environment by creating a local virtualenv in the project and installing the required packages there, which avoids the broken Anaconda OpenMP runtime. I'm trying that inside the workspace first so it doesn't need any extra permissions unless the network blocks the install.

Background terminal finished with python3 -m venv .venv

Background terminal finished with .venv/bin/pip install -r requirements.txt

Background terminal finished with .venv/bin/pip install -r requirements.txt

The dependency install is going through in the project-local virtualenv now. If that finishes cleanly, I'll rerun training/evaluation there and we should have a full verified pipeline rather than just partial smoke coverage.

Background terminal finished with .venv/bin/python - <<'PY' import numpy as np from sklearn.linear_model import LogisticRegression X = np.random.rand(100,5) y = (X[:,0] > 0.5).astype(int) model = LogisticRegression(max_iter=100) model.fit(X,y) print('ok', model.predict_proba(X[:3])) PY

The clean local virtualenv is working, including `scikit-learn` fits. I'm rerunning the pipeline there now so the final outputs are generated and validated in a stable interpreter.

Background terminal finished with .venv/bin/python src/train_model.py

LightGBM itself installed, but its native library can't load on this machine because `libomp` isn't present. I'm broadening the import guard so the trainer cleanly falls back to the scikit-learn backend instead of crashing on startup.

Edited train_model.py
+1 -1

Background terminal finished with .venv/bin/python src/train_model.py

Explored 1 file, 1 list

Background terminal finished with .venv/bin/python src/evaluate_model.py

Background terminal finished with .venv/bin/python src/explainability.py

Background terminal finished with .venv/bin/python src/neo4j_loader.py

I've got the core outputs generated. I'm just cleaning up two rough edges from the smoke run now: a pandas warning in the rolling window code and the matplotlib/font cache warnings during explainability.

Edited common.py
+3 -1

Edited explainability.py
+10 -1

Edited feature_engineering.py
+1 -1

Background terminal finished with .venv/bin/python src/explainability.py

Background terminal finished with .venv/bin/python src/feature_engineering.py

Explored 1 list

Background terminal finished with python3 - <<'PY' import pandas as pd base = '/Users/ashish/Documents/New project/medical_fraud_detection' pred = pd.read_csv(f'{base}/data/processed/predictions.csv') prov = pd.read_csv(f'{base}/data/processed/provider_summary.csv') print('top suspicious claims') print(pred[['claim_event_id','doctor_id','patient_id','fraud_probability','triggered_rules']].sort_values('fraud_probability', ascending=False).head(5).to_string(index=False)) print('\ntop suspicious doctors') print(prov.head(5).to_string(index=False)) PY

Background terminal finished with python3 - <<'PY' import json from pathlib import Path base = Path('/Users/ashish/Documents/New project/medical_fraud_detection') with open(base/'logs'/'training_summary.json') as f: print(f.read()) PY

Background terminal finished with .venv/bin/python - <<'PY' import pandas as pd base = '/Users/ashish/Documents/New project/medical_fraud_detection' pred = pd.read_csv(f'{base}/data/processed/predictions.csv') prov = pd.read_csv(f'{base}/data/processed/provider_summary.csv') print('top suspicious claims')

print(pred[['claim_event_id','doctor_id','patient_id','fraud_probability','triggered_rules']].sort_values('fraud_probability', ascending=False).head(5).to_string(index=False)) print('\ntop suspicious doctors') print(prov.head(5).to_string(index=False)) PY

Background terminal finished with .venv/bin/python - <<'PY' import pandas as pd from pathlib import Path p = Path('/Users/ashish/Documents/New project/medical_fraud_detection/data/processed/claims_with_rules.csv') df = pd.read_csv(p) print('weak rate', df['weak_fraud_label'].mean()) print(df['final_rule_score'].describe()) print(df['triggered_rules'].value_counts().head(10).to_string()) PY

Background terminal finished with .venv/bin/python - <<'PY' import pandas as pd p = '/Users/ashish/Documents/New project/medical_fraud_detection/data/processed/claims_with_graph_features.csv' df = pd.read_csv(p) # compute prospective rule conditions high_claim_cutoff = df['claims_per_doctor'].quantile(0.97) daily_cutoff = df['same_day_doctor_claims'].quantile(0.98) claim_freq_cutoff = df['claim_frequency'].quantile(0.95) cond = (df['claims_per_doctor'] >= high_claim_cutoff) & ((df['same_day_doctor_claims'] >= daily_cutoff) | (df['claim_frequency'] >= claim_freq_cutoff) | (df['patient_provider_overlap'] > 1)) print('new excessive rate', cond.mean()) print('old high doc rate', (df['claims_per_doctor'] >= high_claim_cutoff).mean()) print('daily rate', (df['same_day_doctor_claims'] >= daily_cutoff).mean()) print('claim freq cutoff', claim_freq_cutoff, 'rate', (df['claim_frequency'] >= claim_freq_cutoff).mean()) PY

Background terminal finished with .venv/bin/python - <<'PY' import pandas as pd p = '/Users/ashish/Documents/New project/medical_fraud_detection/data/processed/claims_with_graph_features.csv' df = pd.read_csv(p) print(df['same_day_doctor_claims'].value_counts().head(10).to_string()) print('quantiles', {q: df['same_day_doctor_claims'].quantile(q) for q in [0.9,0.95,0.98,0.99,0.999]}) PY

I found why the weak-label rate was too high: the “too many claims per doctor” rule was effectively flagging every claim because the 98th percentile of same-day doctor claims is 1 in this dataset. I’m making that rule stricter and rerunning so the suspicious set is more realistic.

Edited fraud_rules.py
+2 -1

Background terminal finished with .venv/bin/python src/feature_engineering.py

The label distribution is in a much healthier range now. Before retraining, I’m making the checkpoint resume logic data-aware so old checkpoints from the earlier label definition don’t get reused accidentally.

Edited train_model.py
+17 -2

Background terminal finished with .venv/bin/python src/train_model.py

Explored 1 file, 2 lists

The stricter weak-labeling run is retraining now. It’s slower because the improved labels are producing several genuine AUC improvements across checkpoints, so the early-stopping loop is continuing instead of cutting off at 40 rounds like before.

Background terminal finished with .venv/bin/python src/evaluate_model.py

Background terminal finished with .venv/bin/python src/explainability.py

Background terminal finished with .venv/bin/python src/neo4j_loader.py

Explored 1 file, 1 list

Background terminal finished with .venv/bin/python - <<'PY' import pandas as pd base = '/Users/ashish/Documents/New project/medical_fraud_detection' pred = pd.read_csv(f'{base}/data/processed/predictions.csv') prov = pd.read_csv(f'{base}/data/processed/provider_summary.csv') print('weak rate', pred['weak_fraud_label'].mean()) print('\nTop suspicious claims') print(pred[['claim_event_id','doctor_id','patient_id','fraud_probability','triggered_rules']].sort_values('fraud_probability', ascending=False).head(5).to_string(index=False)) print('\nTop suspicious doctors') print(prov.head(5).to_string(index=False)) PY

Worked for 44m 55s

Implemented the full project in README.md, with the core pipeline in feature_engineering.py, train_model.py, neo4j_loader.py, and the investigator UI in streamlit_app.py. The project now covers data normalization, graph construction, graph features, weak-label fraud rules, checkpointed training/resume, evaluation, explainability, Neo4j export tables, notebooks, config, and setup docs.

I also ran the pipeline end-to-end in a local virtualenv at `/Users/ashish/Documents/New project/medical_fraud_detection/.venv`. Generated outputs are in /Users/ashish/Documents/New project/medical_fraud_detection/data/processed and models/checkpoints in /Users/ashish/Documents/New project/medical_fraud_detection/models. On the current weak labels, the latest smoke test produced a 9.4% suspicious-claim rate and validation metrics of `ROC AUC 0.9991`, `Recall 0.9908`, `F1 0.9931`.

Two practical notes:

- The dataset has no true fraud label, amount, or location fields, so the training target is a rule/graph-based weak label and `claim_amount` is a documented proxy.
- LightGBM support is implemented, but this machine is missing the native `libomp` dependency, so the verified training run used the built-in gradient boosting fallback. Neo4j import CSVs were exported, but I did not load them into a live Neo4j instance because no running server/credentials were available.

To use it now:

- `source /Users/ashish/Documents/New project/medical_fraud_detection/.venv/bin/activate`
- `streamlit run /Users/ashish/Documents/New project/medical_fraud_detection/dashboard/streamlit_app.py`

23 files changed+2967-0

Undo

medical_fraud_detection/.gitignore
medical_fraud_detection/README.md
medical_fraud_detection/config/config.yaml
medical_fraud_detection/config/fraud_rules.cypher
medical_fraud_detection/config/neo4j_schema.cypher
medical_fraud_detection/dashboard/streamlit_app.py
medical_fraud_detection/data/processed/.gitkeep
medical_fraud_detection/data/raw/.gitkeep
medical_fraud_detection/logs/.gitkeep
medical_fraud_detection/models/.gitkeep
medical_fraud_detection/notebooks/EDA.ipynb
medical_fraud_detection/notebooks/graph_analysis.ipynb
medical_fraud_detection/requirements.txt
medical_fraud_detection/src/__init__.py
medical_fraud_detection/src/common.py
medical_fraud_detection/src/data_processing.py
medical_fraud_detection/src/evaluate_model.py
medical_fraud_detection/src/explainability.py
medical_fraud_detection/src/feature_engineering.py
medical_fraud_detection/src/fraud_rules.py
medical_fraud_detection/src/graph_features.py
medical_fraud_detection/src/neo4j_loader.py
medical_fraud_detection/src/train_model.py